

PEDAGOGÍA

Documentación técnica del proyecto

Migración de Google Sheets a un backend propio

(Node.js + Express + SQLite, con despliegue público en Turso y Render)

Autor: José Gerónimo Lugo Flores

Fecha: 13 de julio de 2026

1. Contexto del proyecto

Pedagogía es una plataforma web pensada para conectar a estudiantes y docentes de la preparatoria mediante un test vocacional de opción múltiple. El test evalúa aptitudes (físico-matemático, ciencias biológicas, ciencias sociales, humanidades) y aspectos de personalidad (pensamiento y toma de decisiones, adaptación al entorno, estrés, ansiedad y habilidades sociales), con un total de 55 preguntas distribuidas en 9 secciones.

La primera versión del proyecto usaba Google Sheets como base de datos, conectada mediante un Google Apps Script que recibía las respuestas del formulario y las escribía directamente en una hoja de cálculo.

2. Motivo del cambio

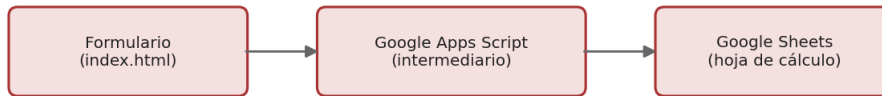
Durante la revisión del proyecto, el profesor indicó que no se permitía utilizar Google Sheets como base de datos del proyecto, ya que el objetivo de la actividad es que el estudiante diseñe, construya y administre su propia base de datos — no que dependa de un servicio externo de terceros que no requiere programación de estructuras de datos, relaciones ni consultas propias.

A partir de esa observación, se rediseñó por completo la capa de almacenamiento del proyecto, reemplazando Google Sheets y Apps Script por un servidor propio con una base de datos relacional real (SQLite), manteniendo el mismo formulario que ya se había construido.

3. Arquitectura: antes y después

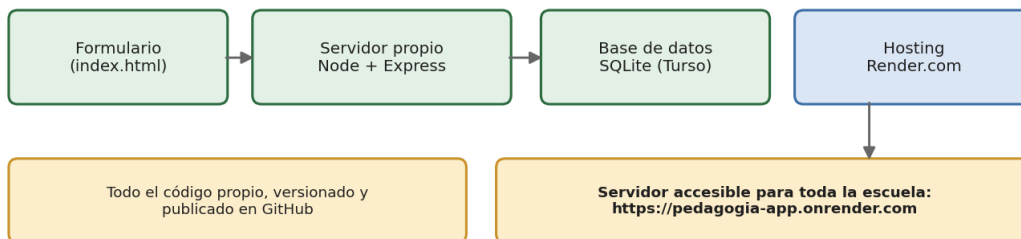
Arquitectura anterior vs. arquitectura actual

ANTES



Rechazado por el profesor: no es una base de datos propia, depende de un servicio externo de terceros.

AHORA



3.1 Arquitectura anterior (rechazada)

- El formulario enviaba las respuestas a un Google Apps Script publicado como aplicación web.
- El Apps Script escribía cada respuesta directamente en una hoja de Google Sheets.
- No existía ninguna base de datos real, ni control sobre el esquema, tipos de datos o relaciones entre tablas.

3.2 Arquitectura actual (aprobada)

- El formulario (index.html) envía las respuestas a un servidor propio construido con Node.js y el framework Express.
- El servidor valida los datos y los guarda en una base de datos SQLite mediante consultas SQL escritas específicamente para este proyecto (creación de tablas, inserciones, consultas de solo lectura).
- Para que la base de datos persista de forma confiable y sea accesible para todos los estudiantes de la preparatoria, la base de datos SQLite se aloja en Turso (un servicio de SQLite en la nube) y el servidor se despliega públicamente en Render.
- El resultado es un sistema donde el estudiante controla el 100% del esquema de la base de datos, las consultas SQL y la lógica del servidor.

4. Esquema de la base de datos

La base de datos cuenta con cuatro tablas. La tabla principal en uso es `respuestas_alumno`, diseñada para coincidir con el formato exacto en el que el formulario envía las respuestas del test (un identificador de alumno y un conjunto de pares pregunta/respuesta).

Tabla	Descripción
respuestas_alumno	Tabla principal: guarda cada respuesta individual del test, con el id del alumno, el identificador de la pregunta, la respuesta dada y la fecha/hora de registro.
estudiantes	Tabla de apoyo para registrar datos generales de un estudiante (nombre, correo, grupo), pensada para una futura versión con perfiles de alumno.
tests	Tabla de apoyo para identificar distintas aplicaciones o versiones del test vocacional a lo largo del tiempo.
respuestas	Tabla de apoyo con un esquema más detallado (estudiante, test y sección), pensada para análisis más granular en versiones futuras.

Estructura de la tabla respuestas_alumno:

```

id          INTEGER PRIMARY KEY AUTOINCREMENT
alumno_id   TEXT NOT NULL
pregunta_id TEXT NOT NULL
respuesta   TEXT NOT NULL
creado_en   TEXT DEFAULT (datetime('now'))

```

5. Endpoints del servidor

El servidor expone las siguientes rutas HTTP, todas construidas con Express:

Método	Ruta	Función
POST	/api/submit	Recibe { alumno_id, respuestas } del formulario y guarda cada respuesta como una fila en respuestas_alumno.
GET	/api/submit/:alumno_id	Devuelve todas las respuestas guardadas de un alumno específico, en formato JSON.
GET	/api/stats	Devuelve un resumen: total de alumnos distintos que han contestado y total de respuestas guardadas.
POST	/api/estudiantes	Registra un estudiante con nombre, correo y grupo (tabla de apoyo).
POST	/api/tests	Registra una nueva aplicación del

Método	Ruta	Función
		test (tabla de apoyo).

6. Paso a paso de la migración

6.1 Preparación del entorno local

1. Instalación de Node.js (incluye npm) en el equipo de desarrollo.
2. Ajuste de la política de ejecución de PowerShell (Set-ExecutionPolicy) para permitir que npm funcione correctamente en Windows.
3. Creación del proyecto backend con las dependencias express, cors y (posteriormente) @libsql/client.

6.2 Construcción del backend (primera versión, local)

4. Diseño del esquema de la base de datos (archivo db.js) con las tablas estudiantes, tests, respuestas y respuestas_alumno.
5. Construcción del servidor Express (server.js) con los endpoints de la sección 5.
6. Pruebas de cada endpoint con curl para verificar que la información se guardaba y se podía consultar correctamente.

6.3 Conexión del formulario existente

7. Localización, dentro del archivo index.html ya construido, de la función que enviaba las respuestas al Google Apps Script.
8. Sustitución de la URL de Apps Script por la ruta del nuevo servidor propio (/api/submit).
9. Corrección de un error de sintaxis en el JavaScript del formulario (una llamada fetch incompleta) detectado mediante la consola de desarrollador del navegador.
10. Corrección de un bloqueo del navegador (Private Network Access) que impedía que la página se comunicara con el servidor local, agregando la cabecera HTTP correspondiente en el servidor.
11. Prueba end-to-end completa: llenado de las 55 preguntas del test desde el navegador y verificación de que las respuestas quedaran guardadas.

6.4 Verificación de los datos

12. Instalación de DB Browser for SQLite para inspeccionar visualmente la base de datos generada (pedagogia.db).
13. Verificación de que las 55 respuestas del test de prueba se guardaron correctamente, cada una con su alumno_id, pregunta_id, respuesta y fecha.
14. Elaboración de un conjunto de consultas SQL de análisis (conteo de alumnos, promedios por sección, distribución de respuestas) para uso posterior con datos reales.

6.5 Control de versiones

15. Instalación de Git y configuración de un repositorio local.
16. Creación de un archivo .gitignore para excluir del repositorio la carpeta node_modules y los archivos de base de datos local.

17. Publicación del código fuente completo en un repositorio de GitHub.

6.6 Migración a un despliegue público

Para que el proyecto pudiera ser utilizado por los estudiantes de toda la preparatoria (y no solo desde la computadora de desarrollo), se realizó una segunda iteración de la arquitectura:

18. Sustitución de la librería de acceso a la base de datos (de better-sqlite3, que solo funciona con un archivo local, a @libsql/client, compatible tanto con un archivo local como con una base de datos alojada en la nube).
19. Creación de una base de datos en Turso (servicio de SQLite en la nube, gratuito para este caso de uso), obteniendo una URL de conexión y un token de acceso.
20. Modificación del servidor para que sirviera también el formulario (index.html) como archivo estático, de manera que el formulario y la API quedaran en la misma dirección web (evitando problemas de CORS).
21. Creación de una cuenta en Render.com y despliegue del repositorio de GitHub como un servicio web gratuito.
22. Configuración de las variables de entorno TURSO_DATABASE_URL y TURSO_AUTH_TOKEN en Render, para que el servidor desplegado se conectara a la base de datos en la nube.
23. Corrección de un error de despliegue (el archivo del formulario tenía un nombre distinto a index.html) y nuevo despliegue automático tras subir la corrección a GitHub.
24. Prueba final desde el enlace público, confirmando que una respuesta enviada desde el navegador se reflejaba correctamente en el panel de datos de Turso.
25. Eliminación de los datos de prueba de la tabla respuestas_alumno en Turso, dejando la base de datos lista para recibir las respuestas reales de los estudiantes.

7. Resultado final

El proyecto Pedagogía cuenta actualmente con:

- Una base de datos relacional propia (SQLite), con un esquema diseñado específicamente para el test vocacional.
- Un servidor propio (Node.js + Express) que expone una API REST documentada en la sección 5.
- **Un despliegue público y accesible desde cualquier dispositivo con conexión a internet, en la dirección:**

<https://pedagogia-app.onrender.com>

- Código fuente versionado y disponible públicamente en GitHub, incluyendo el servidor, el esquema de la base de datos, el formulario y un conjunto de consultas SQL de análisis.

Con esta arquitectura, cada respuesta que un estudiante de la preparatoria envíe a través del formulario se almacena de forma permanente en la base de datos alojada en Turso, quedando disponible para su análisis mediante las consultas SQL incluidas en el proyecto.